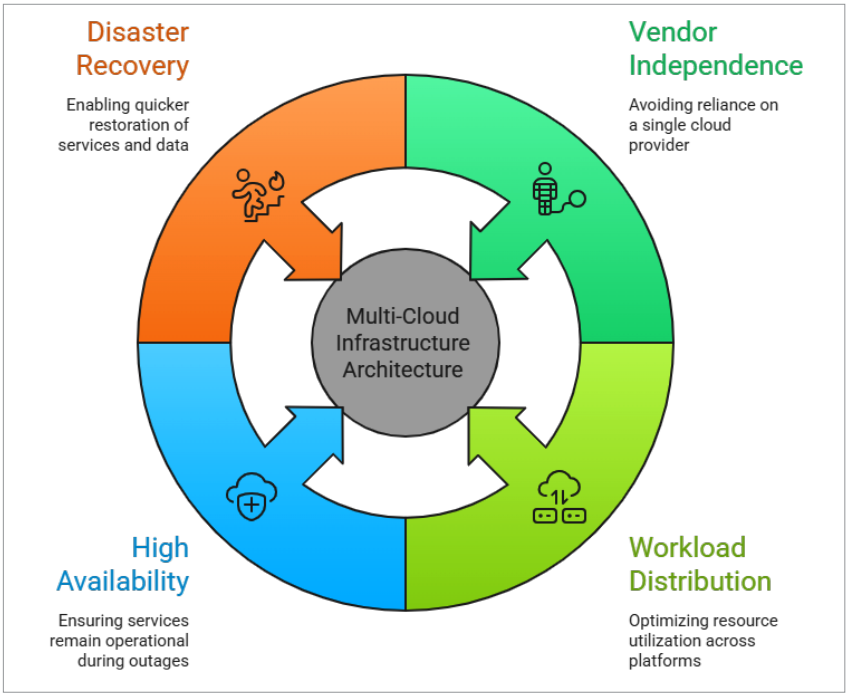

Figure 1: Multi-cloud infrastructure

Why Kubernetes became the foundation of multi-cloud platforms

Kubernetes was developed by Google to manage the deployment of workloads that are packaged as containers. Because of its wide acceptance, it has become the standard for the orchestration of cloud-native infrastructure resources.

One of the strongest features of Kubernetes is its ability to abstract everything that goes into deploying an application on behalf of the user and provide a common API for using the underlying infrastructure. The following key capabilities of Kubernetes facilitate multi-cloud application orchestration:

Deploy once, run anywhere: The flexibility of Kubernetes to deploy applications packaged inside containers allows them to be loaded in both public clouds, private clouds, and any combination of those two environments. The use of one technology provides an identical user experience throughout.

Declarative infrastructure: Kubernetes uses declarative configuration files to define the desired state of an application and its underlying infrastructure, and allows for the automated deployment of both the application and its required infrastructure to achieve this state.

Self-healing: Kubernetes can automatically scale an application. For instance, if you were to lose a container because it died, Kubernetes would automatically create a new replica of that container, providing a more reliable means for a distributed application.

Extensible: Extensibility means the custom resource definitions (CRDs) that come with Kubernetes (as part of the Kubernetes standard distribution) add the new features related to monitoring/managing an application's use of Kubernetes that cannot be achieved using Kubernetes.

While we refer to Kubernetes today as the most popular cloud native orchestrator for multi-cloud, it primarily manages only container workloads (i.e., containers running). To provide for multi-cloud deployment of workloads, additional tools need to be used for controlling (managing) all the resources required for deploying workloads from multiple clouds (i.e., firewalls, load balancers, compute platforms, etc).

The role of open source control planes

A control plane is an essential component for managing infrastructure resources and for guaranteeing a state of the system that is as its intended design. A traditional cloud control plane is designed to work with one service provider and manages compute, network, and storage independently. As an example, Amazon, Google Cloud, and Microsoft Azure use different control planes for virtually all their services.

Open source control planes are designed to serve as a unified layer of infrastructure management across multiple service providers.

The most popular open source control plane solutions available are:

Crossplane: Crossplane builds on top of Kubernetes to allow you to manage cloud resource infrastructure directly from within Kubernetes through the Kubernetes API.

Cluster API: This allows users to declaratively manage their Kubernetes clusters throughout many different infrastructure providers using declarative configuration syntax.

Karmada: Karmada offers central administration of multiple Kubernetes clusters and enables the operation of multiple clusters among service providers.

With these solutions, you can define and manage your infrastructure resources, such as databases, networks, virtual machines, and similar items, as Kubernetes objects.

As a result of this, Kubernetes has evolved from merely being a container orchestration tool into a universal infrastructure control plane.

Crossplane: Kubernetes-native infrastructure management

Among the many popular open source solutions currently available for multi-cloud infrastructure management,